

UNIVERSITY OF DAR ES SALAAM



COLLEGE OF INFORMATION AND COMMUNICATION TECHNOLOGIES (CoICT)

GROUP WORK

Full-Stack Pipeline for Deploying a Self-Hosted LLM Application

COUSE Artificial Intelligence

COUSE CODE IS365

NO	NAME	REG	PROGRAM
1	SAID HAMZA SHISHI	2023-04-12789	BIT
2	VICTOR MANYALA LUGWISHA	2023-04-05600	BIT
3	THOMAS ANDREW SAKAWA	2023-04-12103	BIT
4	CESARIA C MAJIYA	2023-04-06197	BIT
5	GRACE J GALINOMA	2023-04-02198	BIT
6	DAVINA GODWIN RWEIKIZA	2023-04-11912	BIT
7	FRANK FRAYSON MACHA	2023-04-05862	BIT

1. Introduction

This report documents the design, implementation, and testing of the University Student Support Assistant, a full-stack application built. The system integrates a local development setup, a locally hosted LLM, a Django REST API backend, a React frontend, authentication, logging, error handling, and automated testing into a single working pipeline.

2. System Use Case

The assistant was built to help university students get quick answers to common administrative and academic questions without needing to search through multiple web pages or contact different offices. The assistant covers eight core areas of student life:

- Course registration procedures
- Examination rules and regulations
- Library services and borrowing policies
- ICT support and account issues
- Hostel application procedures
- Fee payment methods and deadlines
- Academic calendar and important dates
- Student conduct and disciplinary policy

A student interacts with the system through a chat-style web interface, types a natural language question, and receives an AI-generated answer grounded in a system prompt that defines the assistant's role and scope.

3. Tools and Technologies Used

Component	Technology Used	Purpose
Operating System	Windows	Development environment
Code Editor	Visual Studio Code	Writing and editing all source code
Backend Language	Python 3.10+	Server-side logic
Backend Framework	Django + Django REST Framework	API routing, request handling, authentication
Virtual Environment	Python venv	Isolated dependency management
Local LLM Runtime	Ollama	Serving the language model locally
Model	llama3.2:1b	Generating natural language answers
Frontend Framework	React	User interface and chat experience
Authentication	DRF Token Authentication	Securing the /ask/ endpoint
API Testing	Python requests, Postman	Automated and manual endpoint verification
Version Control	Git and GitHub	Source code management and collaboration

4. System Architecture

The system follows a layered client-server architecture. A student interacts with the React frontend in their browser. The frontend sends an HTTP request containing the student's question to the Django backend. Django authenticates the request using a token, validates the input, and forwards the question (combined with a system prompt) to the locally running Ollama service. Ollama runs the llama3.2:1b model and returns a generated answer. Django logs the interaction, formats the response, and sends it back to the frontend, which displays it in the chat window.

Logical flow: User → Frontend (React) → Backend (Django REST API) → Local LLM (Ollama) → Generated Response → Backend → Frontend → User.

The architecture also includes a configuration layer (Django settings for CORS, authentication, and the Ollama connection details), a logging layer (writing every interaction to a file), and a dedicated error-handling layer that intercepts connection failures, timeouts, and invalid input before they reach the user as raw errors.

5. Implementation Steps

5.1 Environment Setup

A Python virtual environment was created and activated to isolate project dependencies from the system-wide Python installation. Required libraries were installed using pip, including Django, Django REST Framework, django-cors-headers, drf-spectacular, requests, and python-dotenv.

```
PS C:\Users\hp> cd student-support-llm
PS C:\Users\hp\student-support-llm> python -m venv venv
PS C:\Users\hp\student-support-llm> venv\Scripts\activate
(venv) PS C:\Users\hp\student-support-llm> |

Downloading idna-3.18-py3-none-any.whl (65 kB)
Downloading urllib3-2.7.0-py3-none-any.whl (131 kB)
Using cached python_dotenv-1.2.2-py3-none-any.whl (22 kB)
Using cached asgiref-3.11.1-py3-none-any.whl (24 kB)
Downloading certifi-2026.6.17-py3-none-any.whl (133 kB)
Using cached sqlparse-0.5.5-py3-none-any.whl (46 kB)
Using cached tzdata-2026.2-py2.py3-none-any.whl (349 kB)
Installing collected packages: urllib3, tzdata, sqlparse, python-dotenv, idna, charset-normalizer, certifi, asgiref, requests, django, djangorestframework, django-cors-headers
Successfully installed asgiref-3.11.1 certifi-2026.6.17 charset-normalizer-3.4.7 django-6.0.6 django-cors-headers-4.9.0 djangorestframework-3.17.1 idna-3.18 python-dotenv-1.2.2 requests-2.34.2 sqlparse-0.5.5 tzdata-2026.2 urllib3-2.7.0

[notice] A new release of pip is available: 26.1.1 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip
(venv) PS C:\Users\hp\student-support-llm> |
```

```
(venv) PS C:\Users\hp\student-support-llm> pip list
Package            Version
-----
asgiref            3.11.1
certifi            2026.6.17
charset-normalizer 3.4.7
Django             6.0.6
django-cors-headers 4.9.0
djangorestframework 3.17.1
idna               3.18
pip                26.1.1
python-dotenv      1.2.2
requests           2.34.2
sqlparse           0.5.5
tzdata             2026.2
urllib3            2.7.0
(venv) PS C:\Users\hp\student-support-llm> |
```

5.2 Local LLM Installation

Ollama was installed and the llama3.2:1b model was pulled using the ollama pull command. The model was tested directly through Ollama's built-in API on port 11434 before any backend code was written, to confirm the model was functioning correctly in isolation.

```
Microsoft Windows [Version 10.0.26200.8655]
(c) Microsoft Corporation. All rights reserved.

C:\Users\hp>ollama --version
ollama version is 0.30.11

C:\Users\hp>ollama pull llama3.2:1b
pulling manifest
pulling 74701a8c35f6: 100% 1.3 GB
pulling 9664e95ca6a6: 100% 1.4 KB
pulling fcc5a6bec9da: 100% 7.7 KB
pulling a70ff7e570d9: 100% 6.0 KB
pulling 4f659a1e86d7: 100% 485 B
Verifying sha256 digest
writing manifest
success

C:\Users\hp>ollama run llama3.2:1b
>>> hello, how you
" Hello. How can I assist you today?

>>> |Send a message (/? for help)
```

5.3 Django Backend Development

A Django project was created using django-admin startproject, with a dedicated api app handling all business logic. Settings were configured to enable CORS for the React frontend, token-based authentication, structured logging to a file, and the Ollama connection URL and model name. Two main groups of endpoints were implemented: authentication endpoints (register, login, logout, current user) and the core assistant endpoint (ask question), alongside a public health check endpoint.

The llm_client.py module encapsulates all communication with Ollama. It constructs a system prompt that defines the assistant's role and scope, sends the combined prompt to Ollama's generate endpoint, and translates connection or timeout failures into clear, user-friendly exceptions that the view layer can handle gracefully.

```
(venv) PS C:\Users\hp\student-support-llm\backend\config> python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
June 27, 2026 - 17:12:28
Django version 6.0.6, using settings 'config.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

WARNING: This is a development server. Do not use it in a production setting. Use a production WSGI or ASGI server instead.
For more information on production servers see: https://docs.djangoproject.com/en/6.0/howto/deployment/
```

5.4 React Frontend Development

The frontend was built using create-react-app and structured into reusable components: an authentication page handling sign up and login, a chat box component managing the conversation state and API calls, and supporting components for message bubbles and loading indicators. The frontend stores the authentication token returned by the backend and attaches it as an Authorization header on every subsequent request to the protected ask endpoint.

5.5 Authentication Layer

To move beyond a prototype-level system, token-based authentication was added using Django REST Framework's built-in Token Authentication. Students register with a first name, last name, email, and password; the backend creates a Django user object and issues a unique token. This token must be included in the Authorization header of every request to the protected /api/ask/ endpoint, ensuring that only logged-in students can use the assistant.

5.6 Connecting the Full Pipeline

With the backend and frontend running on separate ports (8000 and 3000 respectively), CORS headers were configured to allow cross-origin requests between them. The .env file in the frontend stores the backend API base URL, allowing the connection point to be changed easily without modifying source code.

6. Testing and Results

Three independent testing methods were used to verify the system end-to-end.

6.1 Automated Python Test Script

A script located at tests/test_api.py uses the requests library to automatically test five scenarios: the health check endpoint, a valid question returning a meaningful answer, an empty question correctly rejected with a 400 error, a missing field correctly rejected, and multiple sequential questions to confirm system stability under repeated use. The script authenticates first by registering or logging in a test account and attaching the resulting token to subsequent requests.

```
=====
SETUP: Authenticate test account
PASS - Test account registered and token received

=====
TEST 1: Health Check
PASS - Health returns 200 OK with status: ok

=====
TEST 2: Valid Question
PASS - Got answer (516 chars)
Answer preview: To register for your course, follow these steps:
1. Check the course registration deadline on the university's website or in the student portal.
2. G...

=====
TEST 3: Empty Question
PASS - Empty question returns 400: Please enter a question before submitting.

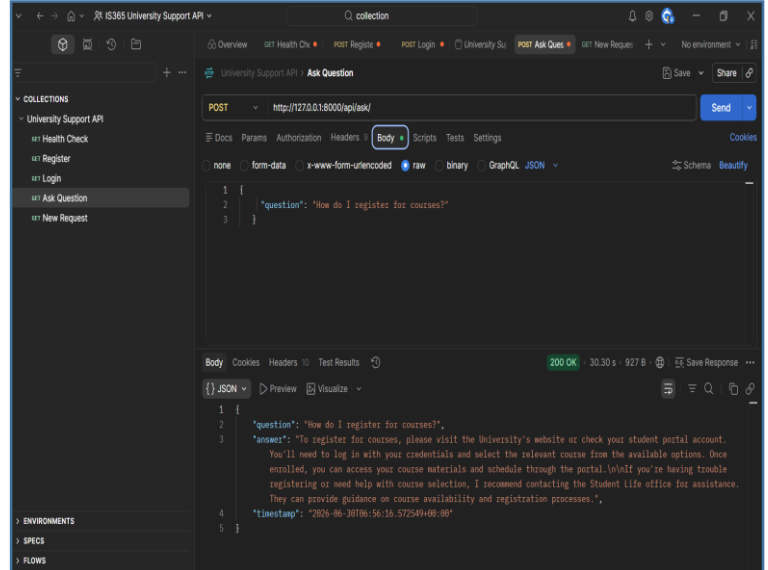
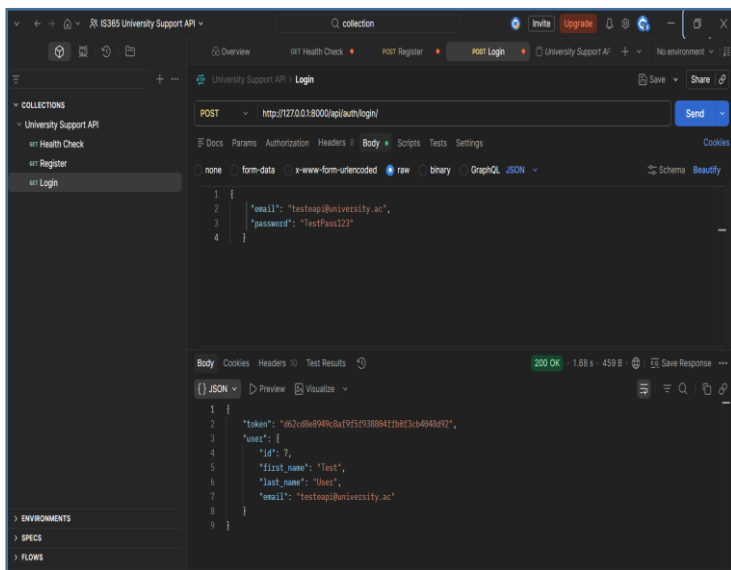
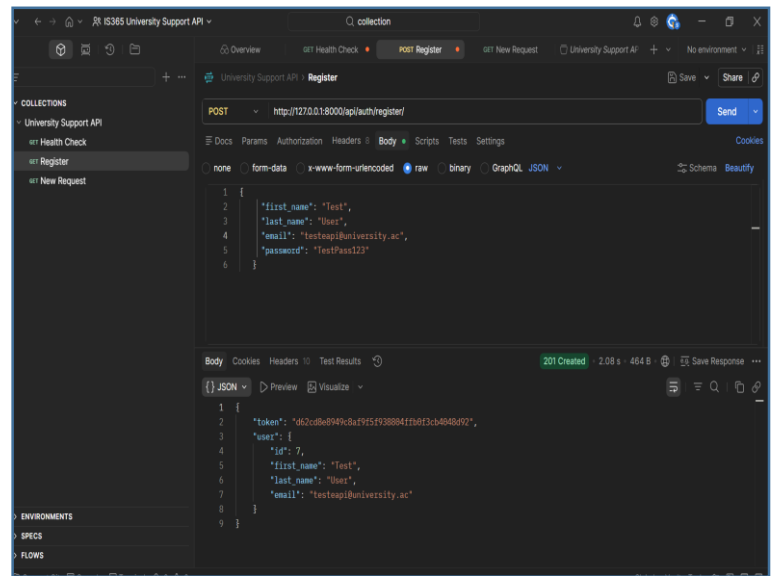
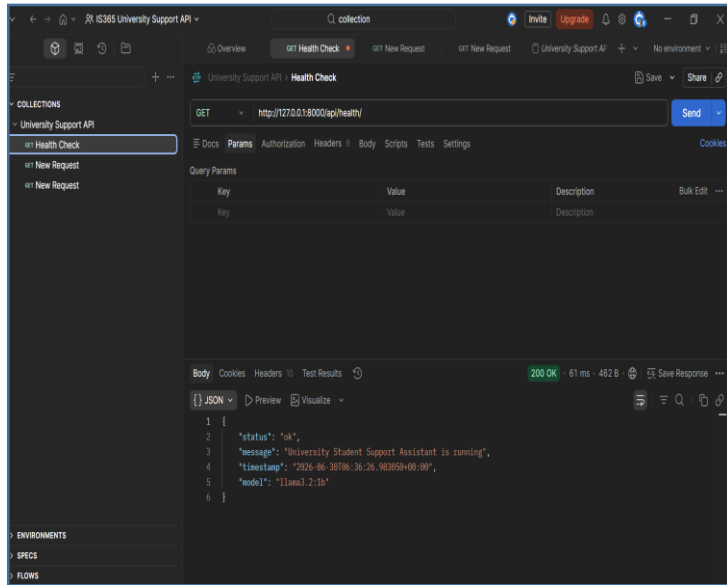
=====
TEST 4: Missing field
PASS - Missing field returns 400

=====
TEST 5: Multiple Questions
PASS - what are the library hours?...
PASS - How do I pay fees?...
PASS - what are exam rules?...

Tests complete.
(venv) PS C:\Users\hp\student-support-llm\backend\config>
```

6.2 Postman Collection

A Postman collection was created containing requests for every endpoint, including deliberate failure cases (empty question, missing authentication) to confirm the system responds with appropriate error codes rather than crashing. This collection is included in the project repository for reproducible manual testing.



7. Challenges Encountered

Adding token authentication initially broke the existing automated test script, since the script was not sending an Authorization header; this was fixed by adding a setup step that registers or logs in a test account before running the protected endpoint tests. A Git-related issue also occurred where the frontend folder was mistakenly tracked as a nested Git repository, causing it to appear as a broken link rather than normal files in the GitHub repository; this was resolved by removing the nested .git folder and re-adding the frontend files to the main repository.

8. Production Readiness Discussion

8.1 Main Components of the Deployed System

The deployed system consists of five core components: the React frontend providing the user interface, the Django REST API backend handling routing and business logic, the LLM client module mediating communication with Ollama, the Ollama runtime hosting the llama3.2:1b model, and a logging and error-handling layer recording every interaction and failure.

8.2 Role of Each Component

Django is central to this pipeline because it provides structured URL routing, built-in authentication tools, middleware for cross-origin requests, and a REST framework that converts Python logic into clean JSON API responses. The LLM model is the system's intelligence layer; it interprets the student's natural language question, combined with a system prompt, and produces a relevant response. The frontend is responsible for collecting input, displaying responses in a readable conversational format, and communicating system state such as loading and error conditions to the user.

8.3 Local Hosting Vs External API

Running the model locally through Ollama means no internet connection is required for inference, there is no per-request cost, and student data never leaves the institution's infrastructure, which is significant for privacy. An external API such as OpenAI's would likely produce higher-quality responses and faster inference on capable hardware, but introduces ongoing costs and requires sending student data to a third-party provider.

8.4 Security Risks in an Organizational Deployment

If deployed within a university without further hardening, several risks exist: log files currently store question text in plain, unencrypted form; the Django secret key is stored directly in source code rather than an environment variable; the system runs over unencrypted HTTP rather than HTTPS; and there is no rate limiting, leaving the API vulnerable to being overwhelmed by automated requests.

8.5 Improvements Required Before Production

Before any real deployment, the system would require HTTPS encryption for all traffic, secrets moved to environment variables, a production-grade database such as PostgreSQL replacing SQLite, a production WSGI server such as Gunicorn behind a reverse proxy such as Nginx, request rate limiting, and stricter input sanitisation to prevent prompt injection attacks against the LLM.

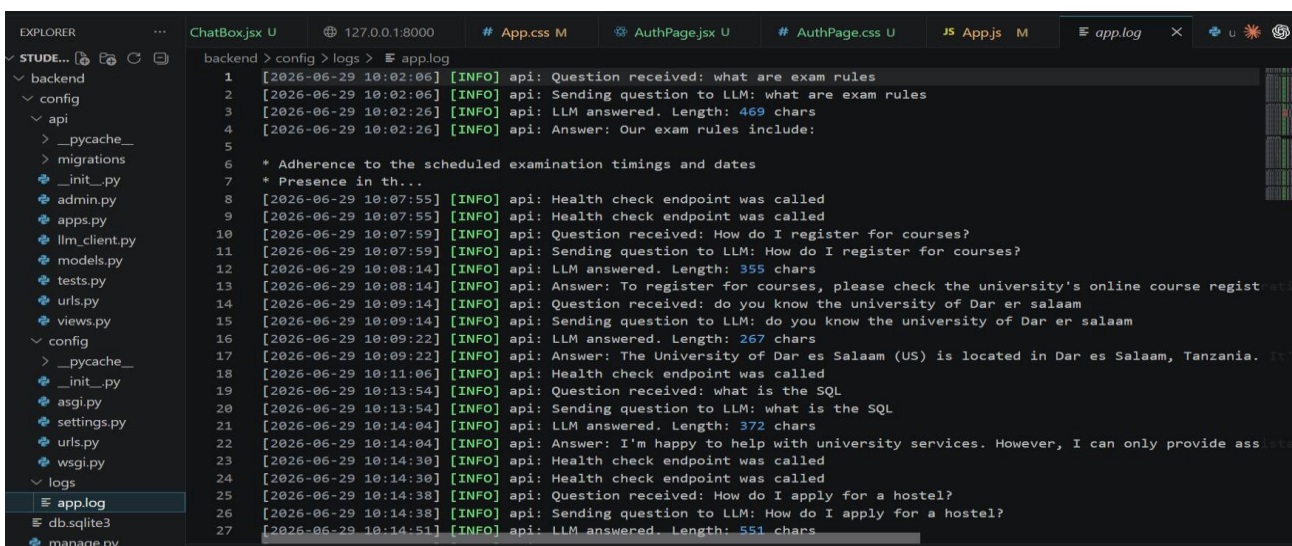
8.6 Monitoring and Data Protection

In real-world use, the system should be monitored using structured log aggregation, response time tracking, and alerting when the Ollama service becomes unreachable. To protect sensitive student information, logs should be encrypted at rest, access to log files restricted to system administrators, and a clear data retention and deletion policy established so that student queries are not stored indefinitely.

9. Conclusion

This project successfully demonstrates a complete, working pipeline for deploying a self-hosted LLM application, from local model serving through to a secured, tested, and documented web interface. While the assistant itself uses a lightweight model and a relatively simple prompt design, the surrounding infrastructure, authentication, error handling, logging, and the three independent testing methods, reflects the kind of engineering discipline required in production AI systems. The exercise reinforced that building a useful AI application depends far less on the intelligence of the model itself and far more on the reliability of the system built around it.

Appendix G — Logging



```
1 [2026-06-29 10:02:06] [INFO] api: Question received: what are exam rules
2 [2026-06-29 10:02:06] [INFO] api: Sending question to LLM: what are exam rules
3 [2026-06-29 10:02:26] [INFO] api: LLM answered. Length: 469 chars
4 [2026-06-29 10:02:26] [INFO] api: Answer: Our exam rules include:
5
6 * Adherence to the scheduled examination timings and dates
7 * Presence in th...
8 [2026-06-29 10:07:55] [INFO] api: Health check endpoint was called
9 [2026-06-29 10:07:55] [INFO] api: Health check endpoint was called
10 [2026-06-29 10:07:59] [INFO] api: Question received: How do I register for courses?
11 [2026-06-29 10:07:59] [INFO] api: Sending question to LLM: How do I register for courses?
12 [2026-06-29 10:08:14] [INFO] api: LLM answered. Length: 355 chars
13 [2026-06-29 10:08:14] [INFO] api: Answer: To register for courses, please check the university's online course regist...
14 [2026-06-29 10:09:14] [INFO] api: Question received: do you know the university of Dar es salaam
15 [2026-06-29 10:09:14] [INFO] api: Sending question to LLM: do you know the university of Dar es salaam
16 [2026-06-29 10:09:22] [INFO] api: LLM answered. Length: 267 chars
17 [2026-06-29 10:09:22] [INFO] api: Answer: The University of Dar es Salaam (US) is located in Dar es Salaam, Tanzania. IT...
18 [2026-06-29 10:11:06] [INFO] api: Health check endpoint was called
19 [2026-06-29 10:13:54] [INFO] api: Question received: what is the SQL
20 [2026-06-29 10:13:54] [INFO] api: Sending question to LLM: what is the SQL
21 [2026-06-29 10:14:04] [INFO] api: LLM answered. Length: 372 chars
22 [2026-06-29 10:14:04] [INFO] api: Answer: I'm happy to help with university services. However, I can only provide ass...
23 [2026-06-29 10:14:30] [INFO] api: Health check endpoint was called
24 [2026-06-29 10:14:30] [INFO] api: Health check endpoint was called
25 [2026-06-29 10:14:38] [INFO] api: Question received: How do I apply for a hostel?
26 [2026-06-29 10:14:38] [INFO] api: Sending question to LLM: How do I apply for a hostel?
27 [2026-06-29 10:14:51] [INFO] api: LLM answered. Length: 551 chars
```

Appendix D — Frontend

localhost:3000

Create account

Log in

START FOR

Create new account.

Already a member? [Log in](#)

First name

Said

Last name

Shishi

Email

said@university.ac

Password

Clear form

Create account

or continue with

Google

GitHub

Student ID

University Student Support

Ask questions about university services, rules, and procedures

js

juma said
juma@gmail.com

Log out

Connected — Welcome, juma

Hi juma

University Assistant

Ask anything about university services, regulations, fees, registration and more.

How do I register for courses?

What are the exam rules?

How do I apply for a hostel?

How do I pay my fees?

Ask about registration, exams, fees, library...

Send

University Student Support

Ask questions about university services, rules, and procedures

js

juma said
juma@gmail.com

Log out

Connected — Welcome, juma

You

What are the exam rules?
12:27 PM

Assistant

For exam rules, we have several policies that apply to all students, including:

- * Exam timetables: You must attend your scheduled exam time. Late arrivals will incur penalties.
- * Exam invigilation: The exam invigilator is responsible for supervising and marking exams.
- * Examination conditions: Exams are held in a quiet and distraction-free environment. Mobile phones and electronic devices should be switched off.
- * Plagiarism and cheating: Strict penalties apply to any form of academic misconduct.

For more information on exam rules, please refer to the University's Exam Handbook or

Ask about registration, exams, fees, library...

Send